

UNIVERSITATEA DIN CRAIOVA

FACULTATEA DE ȘTIINȚE

SPECIALIZAREA METODE ȘI MODELE ÎN

INTELIGENȚA

ARTIFICIALĂ

LUCRARE DE DISERTAȚIE

Îndrumător științific:

Prof. univ. dr. Mirel Coșulschi

Absolvent:

MACAMETE Petre Leonard

CRAIOVA

2026

UNIVERSITATEA DIN CRAIOVA

FACULTATEA DE ȘTIINȚE

SPECIALIZAREA METODE ȘI MODELE ÎN

INTELIGENȚA

ARTIFICIALĂ

Sistem de eLearning bazat pe blockchain

Îndrumător științific:

Prof. univ. dr. Mirel Coșulschi

Absolvent:

MACAMETE Petre Leonard

CRAIOVA

2026

Cuprins

1	Introducere	4
1.1	Contextul lucrării	4
1.2	Motivația cercetării	5
1.3	Problema abordată	6
1.4	Obiective	6
1.5	Contribuții	7
1.6	Structura lucrării	8
	Partea I	9
2	Stadiul cunoașterii în eLearning și Web3	10
2.1	Evoluția aplicațiilor de eLearning	10
2.2	Principii de proiectare pentru experiența educațională	10
2.3	Gamificare și motivație	11
2.4	Modelul learn-to-earn	12
2.5	Aplicații hibride Web2/Web3	12
2.6	Tehnologii imersive și învățare socială	14
2.7	Concluzii ale analizei	14
3	Fundamente tehnologice și justificarea alegerilor	15
3.1	Criterii de selecție	15
3.2	Rust pentru backend	15
3.3	Actix Web pentru API	16
3.4	PostgreSQL și Diesel	16
3.5	Stocare S3 și worker pentru conținut multimedia	17
3.6	Next.js pentru frontend	17
3.7	Ethereum, Solidity și OpenZeppelin	18
3.8	Arhitectură modulară monolitică	18

3.9	Comparație sintetică	19
3.10	Concluzie	19
Partea a II-a		21
4	Analiza și arhitectura sistemului RustLearn	22
4.1	Descriere generală	22
4.2	Actorii sistemului	22
4.3	Cerințe funcționale.....	23
4.4	Cerințe nefuncționale	23
4.5	Arhitectura de ansamblu	24
4.6	Straturile aplicației.....	24
4.7	Modelul de permisiuni	25
4.8	Fluxul de recompensare	26
4.9	Persistența datelor	27
4.10	Valoarea arhitecturii	27
4.11	Concluzie	28
5	Implementarea platformei	29
5.1	Pornirea aplicației și compunerea stării.....	29
5.2	API-ul și middleware-ul	30
5.3	Identitate, parole și verificare email	30
5.4	Modelul cursurilor și al progresului	30
5.5	Organizații și aplicații de profesor	31
5.6	Recompense și politici	31
5.7	Portofele și tranzacții	32
5.8	Contractele inteligente	32
5.9	Worker, procesare video și indexare.....	33
5.10	Frontend-ul produsului	33
5.11	Notificări și feedback operațional	33
5.12	Concluzie	34
6	Validare, operare și limitări	35
6.1	Strategia de validare	35
6.2	Teste backend	35

6.3	Migrații și schema bazei de date	36
6.4	Health și readiness	36
6.5	Docker Compose și mediu local	37
6.6	Securitate și control al riscurilor	37
6.7	Limitări ale implementării	38
6.8	Scenarii de testare recomandate	38
6.9	Concluzie	39
7	Concluzii și direcții viitoare	40
7.1	Sinteza lucrării	40
7.2	Contribuții realizate	40
7.3	Răspuns la problema inițială	41
7.4	Limitări	41
7.5	Direcții viitoare	42
7.6	Concluzie finală	42
	Bibliografie	43

Capitolul 1

Introducere

1.1 Contextul lucrării

Digitalizarea educației a schimbat modul în care sunt proiectate cursurile, modul în care este urmărit progresul și modul în care sunt evaluate competențele. Platformele de eLearning au făcut posibilă distribuirea rapidă a materialelor, organizarea cursurilor în ritm asincron și extinderea accesului la comunități care nu pot participa constant la activități față în față. Totuși, o limitare importantă rămâne felul în care progresul educațional este perceput de cursant. În multe sisteme, rezultatele apar sub forma unor note, bare de progres sau certificate, dar recompensa este întârziată, dificil de verificat independent și rareori integrată într-un mecanism economic transparent.

Tema lucrării, *Sistem de eLearning bazat pe blockchain*, pleacă de la ideea că activitatea educațională poate fi însoțită de un mecanism de recompensare verificabil, fără ca procesul didactic să fie redus la o simplă competiție pentru puncte. Tehnologia blockchain permite înregistrarea unor transferuri și evenimente într-un mod transparent, iar contractele inteligente pot automatiza reguli de emitere, transfer sau verificare a unor jetoane digitale [16, 27]. În același timp, o platformă educațională are nevoie de operații rapide, control al accesului, stocare eficientă a conținutului multimedia, fluxuri administrative și protecția datelor personale. Din acest motiv, soluția analizată în lucrare nu mută toate operațiile în blockchain, ci combină un nucleu web clasic cu un strat de recompense blockchain.

Modelul *learn-to-earn* a devenit vizibil prin platforme care recompensează utilizatorii pentru parcurgerea unor resurse educaționale sau pentru participarea la activități de învățare. Exemple precum BitDegree [14] sau Phemex Learn Crypto [25] arată interesul pentru legătura dintre învățare și recompense digitale, însă aceste soluții sunt frecvent concentrate în jurul conținutului despre criptomonede. Lucrarea propune extinderea acestei logici către o platformă generală

de eLearning, în care cursurile, evaluările, organizațiile, rolurile și recompensele pot funcționa împreună într-un sistem coerent.

1.2 Motivația cercetării

Motivația principală este aceea de a proiecta o platformă în care progresul educațional să fie măsurabil, verificabil și util pentru mai mulți actori: cursanți, profesori, organizații și administratori. Pentru cursant, recompensa poate funcționa ca o confirmare imediată a efortului depus. Pentru profesor, sistemul oferă un mod de a defini activități eligibile pentru recompense și de a valida performanța. Pentru organizații, platforma poate susține programe educaționale interne, bugete de recompensare și rapoarte privind participarea. Pentru administratori, arhitectura trebuie să ofere audit, control al fraudelor, reconciliere și trasabilitate.

Importanța socială a accesului la educație este legată de reducerea unor cercuri de vulnerabilitate. Lipsa educației reduce șansele de angajare, veniturile și participarea socială, iar aceste efecte pot întreține sărăcia și excluziunea. Figura 1.1 ilustrează relația dintre sărăcie, sănătate, educație și capital uman. În acest context, o platformă educațională accesibilă și stimulativă nu rezolvă singură aceste probleme, dar poate contribui la reducerea barierelor prin acces la cursuri, feedback și recunoașterea progresului.

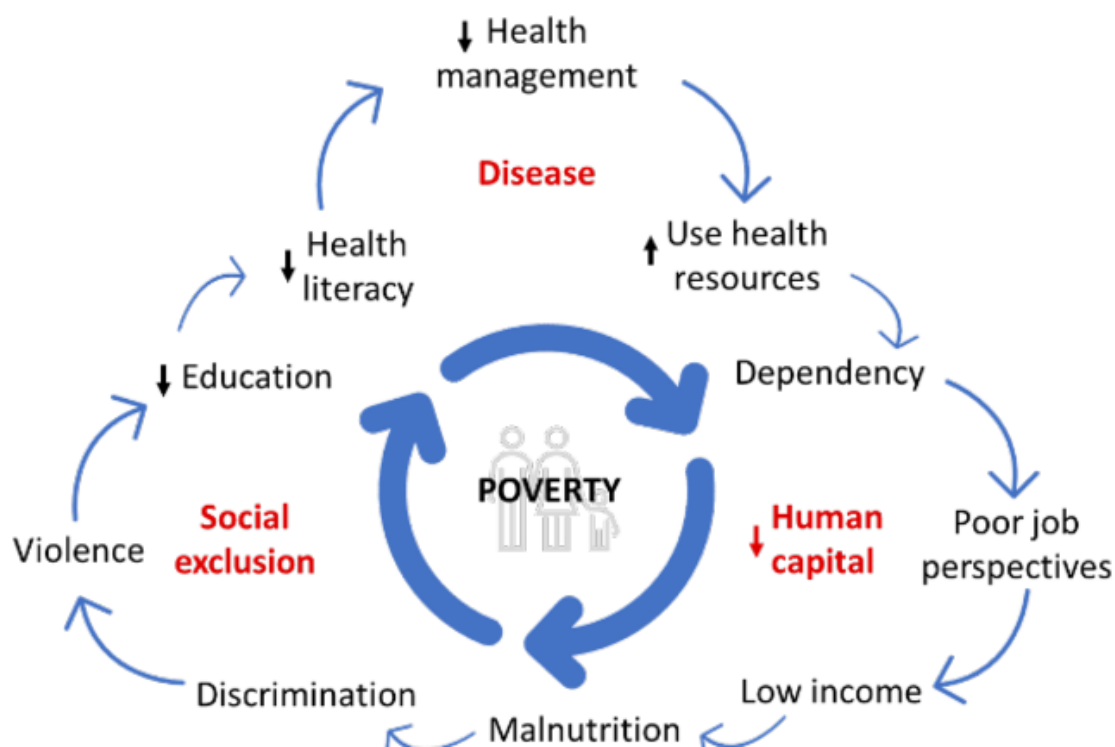


Figura 1.1. Impacturi asociate cercului vicios al sărăciei.¹

Recompensele cu jetoane digitale trebuie tratate cu prudență. Dacă sunt implementate superficial, ele pot încuraja comportamente oportuniste, colectarea artificială de puncte sau dependența de motivație extrinsecă. De aceea, lucrarea pune accent pe un flux de recompensare auditat: un eveniment educațional produce un candidat la recompensă, eligibilitatea este verificată, suma este aprobată, tranzacția este planificată, iar creditarea portofelului intern se face numai după confirmarea etapei relevante. În acest mod, recompensa devine o consecință a progresului educațional durabil, nu un substitut al învățării.

1.3 Problema abordată

Problema analizată constă în proiectarea și implementarea unui sistem de eLearning care să integreze recompense blockchain fără a compromite performanța, securitatea și controlul administrativ necesare unei platforme educaționale. Un sistem exclusiv descentralizat ar oferi transparență, dar ar fi costisitor, lent și dificil de adaptat pentru operații educaționale frecvente. Un sistem exclusiv centralizat ar fi mai simplu de operat, dar ar pierde valoarea auditabilă a transferurilor tokenizate și ar depinde complet de încrederea în operatorul platformei.

Soluția propusă este o arhitectură hibridă. Backend-ul Rust/Actix Web gestionează autentificarea, permisiunile, cursurile, progresul, evaluările, organizațiile și operațiile administrative. PostgreSQL păstrează starea operațională și istoricul necesar raportării. Stocarea compatibilă S3 gestionează fișierele de curs, iar un worker separat procesează materialele video. Contractele Solidity și integrarea Ethereum gestionează LearnToken, importul de tokenuri și transferurile semnate. Frontend-ul Next.js oferă interfețe diferențiate pentru cursanți, profesori, organizații și administratori.

1.4 Obiective

Obiectivul general al lucrării este realizarea unei platforme de eLearning bazate pe blockchain, capabilă să urmărească activitatea educațională și să susțină recompense tokenizate într-un mod controlat. Obiectivele specifice sunt:

- analiza stadiului actual privind eLearning-ul, gamificarea și aplicațiile hibride Web2/Web3;

¹Sursa: Bell, Victoria et al., *Community Health of Children and Adolescents in Sub-Saharan Africa*, European Journal of Medical and Health Sciences, 2023 [1].

- definirea unei arhitecturi care separă domeniul educațional, logica aplicației, infrastructura și interfața HTTP;
- justificarea alegerilor tehnologice pentru backend, frontend, bază de date, stocare, worker și contracte inteligente;
- implementarea unui model de roluri și permisiuni pe trei niveluri: platformă, organizație și curs;
- proiectarea unui flux de recompensare cu stări auditate, reconciliere și protecție împotriva dublării recompenselor;
- integrarea unui token ERC-20 cu suport pentru operații semnate și import de fonduri prin standarde Ethereum;
- validarea soluției prin teste, migrații reversibile, verificări de readiness și scenarii de operare locală.

1.5 Contribuții

Contribuțiile lucrării sunt atât conceptuale, cât și aplicative. Din punct de vedere conceptual, lucrarea arată de ce o arhitectură hibridă este mai potrivită pentru eLearning decât o descentralizare completă. Datele educaționale, permisiunile și procesele administrative rămân într-o bază relațională controlabilă, iar blockchain-ul este folosit pentru valoarea sa specifică: transferuri verificabile, audit public al tokenului și interoperabilitate cu portofele externe. Din punct de vedere aplicativ, lucrarea documentează proiectarea și implementarea platformei RustLearn, cu accent pe separarea responsabilităților, trasabilitate și extensibilitate.

O contribuție importantă este modelarea recompenselor ca proces, nu ca simplă tranzacție. Între finalizarea unei activități de învățare și transferul efectiv al tokenurilor apar verificări, aprobări, stări intermediare și operații de reconciliere. Această abordare reduce riscul de fraudă, permite intervenții administrative și face posibilă explicarea fiecărei decizii. De asemenea, lucrarea evidențiază rolul worker-ului în mutarea operațiilor costisitoare în afara cererilor HTTP, precum procesarea video și indexarea depunerilor de tokenuri.

1.6 Structura lucrării

Lucrarea este organizată conform ghidului de elaborare pentru disertații. Capitolul 1 prezintă tema, motivația, problema abordată, obiectivele și contribuțiile. Partea I analizează stadiul cunoașterii: Capitolul 2 discută eLearning-ul, gamificarea, modelul learn-to-earn și aplicațiile hibride Web3, iar Capitolul 3 justifică tehnologiile utilizate și compară alternativele relevante. Partea a II-a prezintă rezultatele aplicative: Capitolul 4 descrie analiza și arhitectura sistemului RustLearn, Capitolul 5 detaliază implementarea backend-ului, frontend-ului, worker-ului și contractelor inteligente, iar Capitolul 6 descrie testarea, operarea și limitările. Capitolul 7 sintetizează concluziile și direcțiile de dezvoltare viitoare.

Partea I

Capitolul 2

Stadiul cunoașterii în eLearning și Web3

2.1 Evoluția aplicațiilor de eLearning

Aplicațiile de eLearning au evoluat de la simple depozite de fișiere și teste online către sisteme complexe care gestionează parcursuri educaționale, progres, evaluări, comunicare, notificări și rapoarte. Într-o platformă modernă, experiența de învățare nu este definită doar de conținut, ci și de felul în care utilizatorul primește feedback, își vede progresul, revine la materialele relevante și interacționează cu profesorii sau cu organizația care oferă cursul. De aceea, proiectarea aplicațiilor de eLearning trebuie analizată simultan din perspectivă pedagogică, tehnică și operațională.

Un sistem educațional digital trebuie să rezolve mai multe nevoi. Cursantul are nevoie de acces rapid la lecții, de navigare clară, de feedback și de un mod de a înțelege ce trebuie făcut în continuare. Profesorul are nevoie de instrumente de creare și publicare a conținutului, de evaluare, de observare a progresului și de moderare. Organizațiile au nevoie de administrarea membrilor, cursurilor și rapoartelor. Administratorii platformei au nevoie de control asupra rolurilor, auditului, securității și integrărilor. Aceste cerințe duc la o concluzie importantă: eLearning-ul nu poate fi tratat ca o simplă interfață web, ci ca un sistem socio-tehnic cu fluxuri de lucru diferite.

2.2 Principii de proiectare pentru experiența educațională

O interfață de eLearning trebuie să fie ușor de înțeles, previzibilă și accesibilă. Principiile UI/UX discutate în literatura de specialitate pun accent pe claritatea navigării, reducerea efortului cognitiv și coerența componentelor vizuale [22, 24]. Într-un context educațional, aceste principii au o miză suplimentară: dacă interfața este confuză, cursantul consumă energie pentru orientare, nu pentru învățare. Astfel, ecranul principal trebuie să indice cursurile active, lecțiile următoare, statusul

evaluărilor și eventualele notificări importante.

Personalizarea este un alt principiu relevant. Platforma trebuie să afișeze informații diferite pentru un cursant, un profesor, un administrator de organizație sau un operator de platformă. Personalizarea nu înseamnă doar preferințe vizuale, ci adaptarea suprafeței de lucru la permisiunile reale ale utilizatorului. Dacă un profesor are acces la un curs, interfața trebuie să afișeze conținutul, înscrierile și coada de recompense pentru acel curs. Dacă utilizatorul are doar rol de cursant, aceeași platformă trebuie să ascundă operațiunile administrative și să evidențieze lecțiile, progresul și istoricul recompenselor.

Un design eficient pentru eLearning trebuie să mențină echilibrul dintre densitatea informației și claritatea acțiunilor. Platformele educaționale sunt folosite repetat, iar utilizatorii au nevoie de interfețe stabile, cu filtre, stări goale explicite, mesaje de eroare inteligibile și confirmări pentru operații cu efect permanent. Aceste principii sunt relevante pentru RustLearn, deoarece sistemul include rute separate pentru cursanți, profesori, organizații și administratori, fiecare cu propriile decizii și responsabilități.

2.3 Gamificare și motivație

Gamificarea reprezintă utilizarea elementelor inspirate din jocuri în contexte care nu sunt jocuri. Deterding et al. definesc gamificarea ca folosire a elementelor de design de joc în contexte non-ludice [4]. În eLearning, aceste elemente pot include puncte, niveluri, badge-uri, streak-uri, clasamente, provocări și feedback imediat. Scopul nu este transformarea cursului într-un joc, ci crearea unor repere motivaționale care să ajute cursantul să continue, să observe progresul și să primească recunoaștere.

Mecanismele de recompensare trebuie proiectate atent. Un sistem bazat exclusiv pe puncte poate produce comportamente superficiale, în care utilizatorul optimizează pentru recompensă, nu pentru înțelegere. Cercetările privind învățarea online arată importanța angajamentului, a participării și a feedback-ului în menținerea activității educaționale [10, 9, 6]. De aceea, recompensele trebuie legate de evenimente educaționale durabile: finalizarea unei evaluări, promovarea unui test, parcurgerea verificabilă a unui curs sau aprobarea manuală a unei activități relevante.

Figura 2.1 prezintă relația dintre activitatea de învățare și recompensa tokenizată. Elementul central este verificarea. Activitatea produce dovezi, dovezile sunt evaluate prin reguli și prin decizii autorizate, iar recompensa este acordată numai dacă rezultatul este valid. Astfel, gamificarea nu înlocuiește evaluarea, ci o face mai vizibilă și mai motivantă.

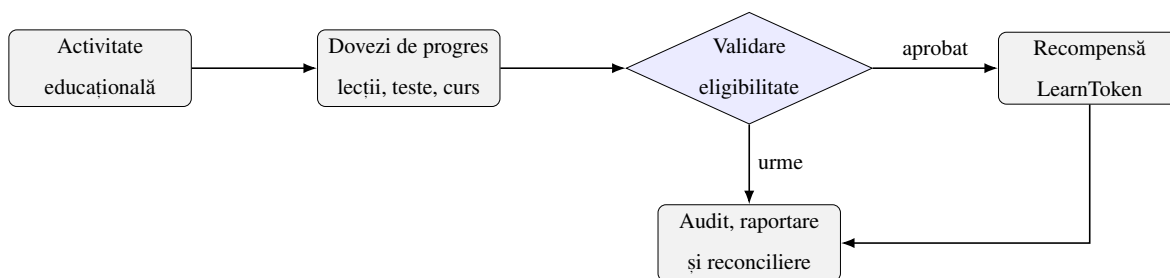


Figura 2.1. Legătura dintre învățare, validare și recompensă tokenizată (elaborare proprie).

2.4 Modelul learn-to-earn

Modelul *learn-to-earn* propune recompensarea cursanților pentru activitate educațională verificată. El este apropiat de gamificare, dar introduce o componentă economică explicită. În loc ca recompensa să fie doar un badge intern, aceasta poate lua forma unui jeton digital care are un istoric verificabil și care poate fi legat de portofele, bugete, reguli de transfer sau politici de retragere. Exemplele existente, precum BitDegree și Phemex Learn Crypto, arată că utilizatorii pot fi atrași de cursuri atunci când parcurgerea conținutului are o recompensă clară [14, 25].

Limitarea multor platforme learn-to-earn este concentrarea pe conținut legat de crypto. Această concentrare este firească în fazele inițiale ale ecosistemului, deoarece utilizatorii familiarizați cu portofele și tokenuri sunt mai dispuși să accepte astfel de mecanisme. Totuși, ideea poate fi extinsă la educația STEM, la programe de recalificare, la cursuri organizaționale și la programe de învățare continuă. În aceste cazuri, recompensa nu trebuie văzută ca speculație financiară, ci ca instrument de recunoaștere și stimulare.

Un model matur learn-to-earn trebuie să includă controale împotriva abuzului. Dacă un cursant poate obține recompense repetate pentru același eveniment sau poate produce dovezi artificiale de progres, sistemul își pierde credibilitatea. De aceea, sunt necesare chei de idempotență, politici de recompensare versionate, stări intermediare și mecanisme de blocare pentru profesori, organizații, cursuri sau politici suspecte. Aceste cerințe depășesc simpla gamificare și transformă recompensa într-un proces administrativ și tehnic.

2.5 Aplicații hibride Web2/Web3

O aplicație hibridă combină infrastructura centralizată a aplicațiilor web cu mecanisme descentralizate precum blockchain-ul, contractele inteligente și portofelele crypto. Literatura despre Web3 subliniază ideea de control asupra datelor, proprietate digitală și aplicații descentralizate [8, 21].

În practică, însă, multe aplicații au nevoie de componente Web2 pentru performanță, experiență de utilizare și conformitate operațională. Această combinație este valoroasă în special pentru eLearning, unde majoritatea operațiilor sunt frecvente, sensibile la latență și dependente de permisiuni.

Brave Browser este un exemplu de aplicație hibridă: oferă o experiență web familiară, dar integrează portofel, mecanisme de confidențialitate și model de recompensare prin tokenuri [15, 17]. Valoarea exemplului nu constă în identitatea exactă a produsului, ci în principiul arhitectural: utilizatorul nu trebuie forțat să înțeleagă toate detaliile blockchain pentru a beneficia de funcțiile Web3. În același mod, un cursant RustLearn trebuie să poată urmări lecții și recompense fără ca fiecare acțiune educațională să devină o tranzacție blockchain.

Tabelul 2.1 sintetizează diferențele dintre cele trei variante arhitecturale relevante pentru lucrare.

Tabela 2.1. Comparatie între abordări centralizate, descentralizate și hibride (elaborare proprie).

Criteriu	Sistem centralizat	Sistem descentralizat	Sistem hibrid
Performanță	Foarte bună pentru operații frecvente și interogări complexe	Depinde de rețea, costuri și confirmări	Bună pentru fluxuri educaționale, cu blockchain folosit selectiv
Audit public	Limitat la logurile operatorului	Puternic pentru evenimente on-chain	Puternic pentru tokenuri, completat de audit intern
Cost operațional	Predictibil, mai ales pentru date și fișiere	Costuri pe tranzacții și contracte	Costuri controlate prin alegerea evenimentelor on-chain
Experiență utilizator	Familiară și rapidă	Poate necesita portofel și semnături dese	Familiară, cu opțiuni Web3 când sunt necesare
Control administrativ	Ridicat	Limitat de regulile contractului	Ridicat pentru educație, transparent pentru recompense

2.6 Tehnologii imersive și învățare socială

Pe lângă gamificare și recompense, eLearning-ul modern folosește simulări, realitate augmentată, realitate virtuală și comunități de învățare. Studiile despre realitatea virtuală în educația superioară arată potențialul experiențelor imersive pentru domenii în care practica este greu de reprodus în siguranță sau la scară mare [11, 5]. Simulările și jocurile educaționale pot transforma concepte abstracte în situații aplicabile [12, 3].

În cazul RustLearn, lucrarea nu implementează AR/VR, dar arhitectura ia în calcul conținut multimedia, procesare video și stocare de obiecte. Această alegere este importantă deoarece o platformă de eLearning nu trebuie să fie limitată la text. Lecțiile video, fișierele atașate, resursele audio și materialele interactive pot fi integrate progresiv dacă infrastructura suportă încărcare, procesare și livrare eficientă.

Învățarea socială completează componenta individuală. Forumurile, comentariile, discuțiile moderate și colaborarea între cursanți pot crește angajamentul și pot oferi contexte de aplicare a cunoștințelor [6, 9]. Într-un sistem cu recompense, aceste funcții trebuie moderate cu grijă, deoarece activitatea socială poate deveni sursă de abuz dacă este recompensată fără verificare. De aceea, RustLearn tratează permisiunile, moderarea și auditul ca elemente de bază, nu ca funcții secundare.

2.7 Concluzii ale analizei

Analiza stadiului cunoașterii conduce la trei observații relevante pentru proiectarea platformei. În primul rând, eLearning-ul eficient are nevoie de interfețe clare, feedback și trasee adaptate rolului utilizatorului. În al doilea rând, gamificarea și recompensele pot crește motivația, dar trebuie legate de activități educaționale reale și de verificări antifraudă. În al treilea rând, blockchain-ul aduce valoare atunci când este folosit pentru ceea ce face bine: tokenuri, trasabilitate și interoperabilitate. Pentru cursuri, roluri, fișiere, rapoarte și administrare, o infrastructură web centralizată rămâne mai potrivită. Aceste observații fundamentează arhitectura hibridă prezentată în capitolele următoare.

Capitolul 3

Fundamente tehnologice și justificarea alegerilor

3.1 Criterii de selecție

Alegerea tehnologiilor pentru un sistem de eLearning bazat pe blockchain trebuie să pornească de la cerințele reale ale produsului. Platforma gestionează utilizatori, autentificare, cursuri, conținut, evaluări, organizații, roluri, portofele, recompense și operații de audit. Aceste fluxuri cer consistență, securitate, performanță și posibilitatea de a evolua fără rescriere completă. Din acest motiv, criteriile de selecție folosite în lucrare sunt: siguranța și robustețea codului, suportul pentru operații asincrone, maturitatea ecosistemului, integrarea cu PostgreSQL, compatibilitatea cu stocare de obiecte, capacitatea de a interacționa cu Ethereum și ergonomia pentru dezvoltarea frontend-ului.

Un criteriu esențial este separarea responsabilităților. Tehnologiile nu au fost alese pentru a produce un sistem spectaculos din punct de vedere tehnologic, ci pentru a susține o arhitectură explicabilă. Backend-ul trebuie să fie sursa de adevăr pentru reguli și permisiuni, baza de date trebuie să păstreze istoricul operațional, worker-ul trebuie să proceseze sarcini costisitoare în afara cererilor HTTP, iar blockchain-ul trebuie să gestioneze componentele care beneficiază de audit public.

3.2 Rust pentru backend

Rust a fost ales pentru backend deoarece oferă un echilibru bun între performanță, siguranța memoriei și control asupra erorilor. Limbajul pune accent pe proprietate, împrumuturi și tipuri explicite,

ceea ce reduce o categorie importantă de defecte întâlnite în aplicațiile server: acces invalid la memorie, date mutate accidental sau stări tratate incomplet [20, 7]. Pentru o platformă care lucrează cu autentificare, permisiuni și tranzacții financiare interne, aceste proprietăți sunt relevante.

O alternativă ar fi fost Node.js, care are un ecosistem foarte matur pentru aplicații web și permite dezvoltare rapidă. Totuși, pentru logica de domeniu cu multe tipuri, stări și erori, Rust oferă o disciplină mai mare la compilare. Python sau Ruby ar fi redus costul inițial de dezvoltare, dar ar fi cerut mai multă atenție la validarea tipurilor și la performanță în fluxuri concurente. Java sau Kotlin ar fi fost opțiuni solide pentru backend enterprise, însă Rust oferă o amprentă de runtime mai mică și o integrare foarte bună cu sisteme asincrone moderne.

3.3 Actix Web pentru API

Actix Web a fost ales ca framework HTTP deoarece este matur, performant și integrat cu ecosistemul asincron Rust. Documentația oficială îl prezintă ca framework web pentru servicii HTTP, rutare, extractori și middleware [28]. În RustLearn, Actix Web permite definirea unui API modular, cu rute grupate pe contexte: identitate, cursuri, organizații, recompense, portofele, raportare și operații. Middleware-ul este important deoarece autentificarea JWT și verificările de permisiuni trebuie aplicate consecvent pe rute.

O alternativă posibilă ar fi fost Axum, bazat pe ecosistemul Tower. Axum are o integrare elegantă cu tipurile Rust și este foarte potrivit pentru servicii moderne. Alegerea Actix Web este justificată prin maturitate, exemple existente și potrivirea cu structura proiectului. Pentru lucrare, diferența critică nu este un benchmark între framework-uri, ci posibilitatea de a exprima clar structuri HTTP, extractori, middleware și state partajat.

3.4 PostgreSQL și Diesel

PostgreSQL a fost ales ca bază de date principală deoarece platforma are nevoie de relații complexe, tranzacții, constrângeri, indexuri și interogări pentru rapoarte. Datele educaționale și financiare interne nu sunt documente izolate, ci entități conectate: utilizatori, roluri, organizații, cursuri, înscrieri, progres, evaluări, candidați la recompense, tranzacții interne și evenimente de audit. Documentația PostgreSQL evidențiază suportul pentru un model relațional complet, extensibil și matur [26].

Diesel a fost ales pentru accesul la date deoarece oferă interogări tipizate și migrații in-

tegrate în ecosistemul Rust [18]. Într-un proiect cu multe tabele și multe permisiuni, beneficiul tipurilor este semnificativ: erorile de nume de coloane, tipuri incompatibile sau rezultate așteptate incorect sunt detectate mai devreme. Diesel Async permite menținerea modelului tipizat și în operații asincrone, ceea ce se potrivește cu Actix Web.

O alternativă ar fi fost o bază NoSQL, de exemplu MongoDB. Aceasta ar fi simplificat stocarea unor documente flexibile, dar ar fi complicat integritatea relațiilor dintre utilizatori, roluri, cursuri și tranzacții. Pentru RustLearn, consistența este mai importantă decât flexibilitatea documentelor. O altă alternativă ar fi fost un ORM mai dinamic, dar Diesel este preferabil tocmai pentru că impune o relație clară între schemă și cod.

3.5 Stocare S3 și worker pentru conținut multimedia

Conținutul educațional include materiale text, fișiere și video. Baza de date nu este potrivită pentru stocarea fișierelor mari, iar serverul HTTP nu trebuie să proceseze video în interiorul unei cereri. Din acest motiv, RustLearn folosește stocare compatibilă S3 pentru obiecte și un worker separat pentru procesare. Amazon S3 este un model consacrat de stocare de obiecte, organizat prin bucket-uri și chei, potrivit pentru fișiere distribuite și scalabile [13]. În dezvoltarea locală, RustFS oferă o implementare compatibilă S3.

Worker-ul procesează sarcini precum transcodarea video, extragerea audio, retry-uri și indexarea depunerilor de tokenuri. Această separare este valoroasă deoarece cererile HTTP rămân rapide, iar operațiile costisitoare sunt executate asincron. Dacă transcodarea video ar fi făcută în request, utilizatorul ar aștepta inutil, iar serverul API ar fi expus la blocaje. Un worker dedicat permite limitarea concurenței, monitorizarea cozii și retry-uri controlate.

3.6 Next.js pentru frontend

Frontend-ul a fost realizat cu Next.js deoarece platforma are nevoie de rute clare, componente React, posibilitatea de a separa suprafețele pentru cursanți, profesori, organizații și administratori, precum și de integrare cu API-ul backend. Documentația Next.js evidențiază App Router ca mecanism de rutare bazat pe sistemul de fișiere și pe funcționalități moderne React [30]. În RustLearn, această abordare permite rute precum `/learn`, `/teach`, `/organizations`, `/admin`, `/wallet` și `/rewards`.

O alternativă ar fi fost o aplicație React simplă cu Vite. Aceasta ar fi fost suficientă pentru

o interfață mai mică, dar Next.js oferă structură de rutare și convenții utile pentru un produs cu multe ecrane. O altă alternativă ar fi fost Flutter pentru web, dar ecosistemul React/Next.js este mai potrivit pentru interfețe administrative web dense, formulare, tabele, filtre și integrare rapidă cu API-uri HTTP.

3.7 Ethereum, Solidity și OpenZeppelin

Componenta blockchain folosește Ethereum și contracte Solidity pentru LearnToken. Solidity este limbajul principal pentru contracte inteligente pe EVM [27], iar OpenZeppelin oferă implementări verificate pentru standarde precum ERC-20 [23]. LearnToken extinde ERC-20 cu opțiuni precum burn și permit, iar contractele auxiliare permit operații semnate și import de tokenuri.

Standardul EIP-712 definește semnarea datelor structurate, ceea ce permite utilizatorilor să semneze mesaje mai clare decât octeți opaci [19]. EIP-2612 extinde ERC-20 cu funcția `permit`, prin care o aprobare poate fi realizată prin semnătură, nu printr-o tranzacție separată [29]. Aceste standarde sunt importante pentru experiența utilizatorului, deoarece reduc numărul de pași on-chain și permit platformei să construiască fluxuri în care utilizatorul semnează o intenție, iar execuția poate fi finalizată controlat.

Blockchain-ul nu este folosit pentru toate datele. Notele, lecțiile, profilurile, permisiunile și rapoartele rămân în baza de date. Această decizie este justificată de cost, performanță și protecția datelor. Blockchain-ul este folosit pentru tokenuri, tranzacții și evenimente în care transparența și interoperabilitatea sunt avantaje reale. Astfel, arhitectura evită atât centralizarea completă a recompenselor, cât și descentralizarea inutilă a datelor educaționale.

3.8 Arhitectură modulară monolitică

RustLearn adoptă o arhitectură modulară monolitică. Sistemul rulează ca o aplicație backend principală, dar codul este separat în contexte: `http`, `application`, `domain`, `infra` și `bootstrap`. Această organizare păstrează simplitatea operațională a unui monolit, dar introduce limite interne clare. În sistemele distribuite, împărțirea prematură în servicii poate aduce costuri de comunicare, consistență și observabilitate [2]. Pentru un produs aflat în evoluție, monolitul modular este mai ușor de testat, migrat și refactorizat.

Valoarea acestei arhitecturi constă în faptul că domeniile sunt separate în cod înainte de a fi separate fizic. Dacă în viitor worker-ul de recompense, serviciul de conținut sau indexerul

blockchain trebuie extrase în servicii independente, limitele conceptuale există deja. Până atunci, tranzacțiile locale, migrațiile și testele rămân mai simple.

3.9 Comparație sintetică

Tabelul 3.1 prezintă pe scurt tehnologiile folosite și motivele alegerii lor.

Tabela 3.1. Tehnologii folosite și justificarea alegerii (elaborare proprie).

Componentă	Tehnologie	Justificare
Backend API	Rust și Actix Web	Siguranță la compilare, performanță, middleware, rutare modulară și potrivire cu aplicații asincrone.
Persistență	PostgreSQL și Diesel	Model relațional, tranzacții, migrații, tipare clare pentru date conectate și interogări tipizate.
Media	S3 compatibil și worker	Fișierele mari sunt separate de baza de date, iar procesarea video nu blochează cererile HTTP.
Frontend	Next.js și React	Rutare pe suprafețe de produs, componente reutilizabile și interfețe diferite pentru roluri diferite.
Blockchain	Solidity, Ethereum, OpenZeppelin	Token ERC-20, standarde mature, semnături EIP-712 și permit EIP-2612 pentru fluxuri Web3 controlate.
Operare	Docker Compose și Kubernetes	Aliniere între dezvoltare locală, testare și deploy, cu servicii pentru API, web, DB, S3, Anvil și worker.

3.10 Concluzie

Tehnologiile selectate susțin o arhitectură hibridă în care fiecare componentă are un rol clar. Rust și Actix Web oferă un backend robust, PostgreSQL și Diesel susțin consistența datelor, S3 și worker-ul gestionează conținutul multimedia, Next.js oferă suprafețe de produs moderne, iar Ethereum

adaugă valoare acolo unde tokenizarea și auditul public contează. Alegerea nu urmărește maximizarea descentralizării, ci construirea unui sistem educațional util, sigur și extensibil.

Partea a II-a

Capitolul 4

Analiza și arhitectura sistemului RustLearn

4.1 Descriere generală

RustLearn este platforma aplicativă dezvoltată pentru tema *Sistem de eLearning bazat pe blockchain*. Sistemul combină funcții de eLearning cu mecanisme de recompensare bazate pe LearnToken. Din punct de vedere funcțional, platforma permite autentificarea utilizatorilor, administrarea cursurilor, publicarea conținutului, urmărirea progresului, gestionarea organizațiilor, evaluarea cursanților, administrarea portofelelor și operarea unui flux de recompense. Din punct de vedere tehnic, sistemul este format dintr-un backend Rust/Actix Web, o bază de date PostgreSQL, stocare compatibilă S3, un worker de procesare, contracte Solidity și un frontend Next.js.

Arhitectura a fost proiectată pentru a evita două extreme. Prima extremă este platforma complet centralizată, în care toate recompensele sunt doar înregistrări interne. Aceasta ar fi simplă, dar ar reduce valoarea de transparență și interoperabilitate a tokenului. A doua extremă este platforma complet on-chain, în care fiecare progres, rol sau evaluare ar fi tranzacție blockchain. Aceasta ar fi costisitoare, lentă și nepotrivită pentru date educaționale sensibile. RustLearn folosește o cale intermediară: operațiunile educaționale rămân off-chain, iar tokenurile și tranzacțiile relevante sunt integrate cu Ethereum.

4.2 Actorii sistemului

Sistemul are patru categorii principale de actori. Cursantul consumă conținut, solicită înscrierea la cursuri, parcurge lecții, susține evaluări, urmărește recompensele și își gestionează portofelul. Profesorul creează conținut, administrează cursurile pe care le predă, verifică înscrieri, evaluează studenți și aprobă candidați la recompensă în limita permisiunilor primite. Organizația gestionează

programe educaționale, membri, cursuri sponsorizate, rapoarte și bugete de recompensare. Administratorul platformei gestionează roluri globale, aplicații de profesor, politici de recompensare, blocaje antifraudă, exporturi, KYC, portofele și reconciliere.

Acești actori nu sunt doar etichete de interfață. Ei corespund unor scopuri de permisiuni diferite: platformă, organizație și curs. Această separare este necesară deoarece același utilizator poate fi student într-un curs, profesor în alt curs și membru al unei organizații. Dacă sistemul ar folosi doar roluri globale, ar apărea fie privilegii excesive, fie restricții prea dure. Modelul pe scopuri permite exprimarea unei realități educaționale mai fine.

4.3 Cerințe funcționale

Cerințele funcționale principale sunt:

- autentificare cu token JWT, politici de parolă, verificare email și expunere JWKS pentru verificare externă;
- administrarea cursurilor, capitolelor, conținutului și stărilor de publicare;
- urmărirea progresului și susținerea evaluărilor cu scor și prag de promovare;
- gestionarea organizațiilor, membrilor, cursurilor asociate și rapoartelor;
- flux de aplicare și aprobare pentru profesori;
- portofele interne pentru utilizatori și organizații, cu audit al tranzacțiilor;
- candidați la recompensă, politici de recompensare, aprobări, plăți, creditare wallet și notificări;
- blocaje antifraudă și reconciliere pentru stări inconsistente;
- procesarea fișierelor video prin worker și stocare de obiecte;
- health checks și readiness checks pentru baza de date, S3 și Ethereum.

4.4 Cerințe nefuncționale

Cerințele nefuncționale sunt la fel de importante ca funcțiile vizibile. Platforma trebuie să fie sigură, deoarece gestionează identitate, portofele și permisiuni. Trebuie să fie auditabilă, deoarece

recompensele pot avea valoare economică și pot genera dispute. Trebuie să fie extensibilă, deoarece fluxurile educaționale și regulile de recompensare pot evolua. Trebuie să fie operabilă, adică să poată fi rulată local, testată, migrată și monitorizată. De asemenea, trebuie să fie rezilientă la eșecuri parțiale: o tranzacție blockchain eșuată nu trebuie să producă dublă creditare, iar un eșec de procesare video nu trebuie să blocheze API-ul.

4.5 Arhitectura de ansamblu

Figura 4.1 prezintă arhitectura de ansamblu. Browserul comunică cu aplicația Next.js, iar aceasta consumă API-ul Actix Web. Backend-ul aplică middleware de autentificare și permisiuni, apoi direcționează cererile către use case-uri. Use case-urile lucrează cu reguli de domeniu și adaptoare de infrastructură. PostgreSQL păstrează starea aplicației, S3 păstrează obiectele media, iar Ethereum păstrează tokenul și evenimentele on-chain. Worker-ul preia sarcini din bază de date și interacționează cu S3 și Ethereum fără a bloca cererile utilizatorilor.

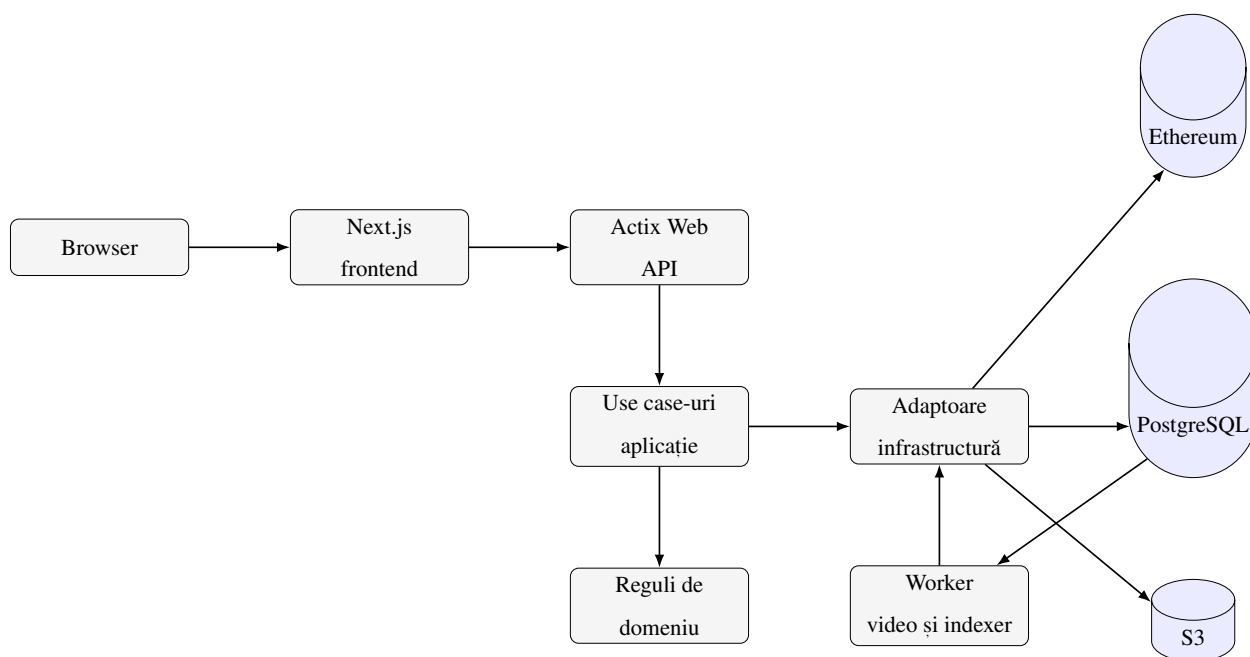


Figura 4.1. Arhitectura generală RustLearn (elaborare proprie).

4.6 Straturile aplicației

Codul backend este separat în straturi. Directorul `src/http` conține rute, DTO-uri, extractor și middleware. Directorul `src/application` conține use case-uri, comenzi, porturi și rezultate. Directorul `src/domain` păstrează vocabularul de domeniu și regulile pure. Directorul

`src/infra` conține adaptoare concrete pentru PostgreSQL, S3, Ethereum, JWT și notificări. Directorul `src/bootstrap` compune aplicația, configurează starea, înregistrează rutele și pornește contractele necesare.

Această separare este valoroasă deoarece împiedică amestecarea logicii de business cu detaliile HTTP. De exemplu, o regulă privind validarea unei recompense nu trebuie să depindă de un handler Actix. Ea trebuie să poată fi testată ca serviciu de aplicație sau regulă de domeniu. La fel, interogările PostgreSQL trebuie încapsulate în adaptoare, nu scrise direct în rute. Rezultatul este un cod mai testabil și mai ușor de modificat.

Tabela 4.1. Responsabilitățile straturilor backend (elaborare proprie).

Strat	Responsabilitate
<code>http</code>	Expune API-ul, validează forma cererilor, aplică middleware și transformă răspunsurile în JSON.
<code>application</code>	Ordonează pașii unui caz de utilizare, verifică porturi, produce rezultate și erori controlate.
<code>domain</code>	Definește reguli și concepte independente de framework: roluri, permisiuni, statusuri, tipuri de recompensă.
<code>infra</code>	Implementează accesul concret la PostgreSQL, S3, Ethereum, JWT, email local și notificări.
<code>bootstrap</code>	Leagă modulele, construiește starea aplicației și configurează pornirea procesului.

4.7 Modelul de permisiuni

Modelul de permisiuni este unul dintre elementele care dau valoare arhitecturii. RustLearn nu se limitează la roluri globale. Există roluri de platformă, roluri de organizație și roluri de curs. De exemplu, un utilizator poate avea permisiunea `VIEW_COURSE` într-un curs, `MANAGE_ORG_MEMBERS` într-o organizație și nicio permisiune administrativă la nivel de platformă. Middleware-urile specializate verifică aceste scopuri înainte ca operațiile sensibile să fie executate.

Figura 4.2 prezintă modelul conceptual. Permisia efectivă este rezultatul combinației dintre identitatea utilizatorului, rolurile directe, rolurile pe organizație, rolurile pe curs și eventualele permisiuni delegate. Acest model permite administrare fină și reduce riscul ca un actor să

primească privilegii mai largi decât sunt necesare.

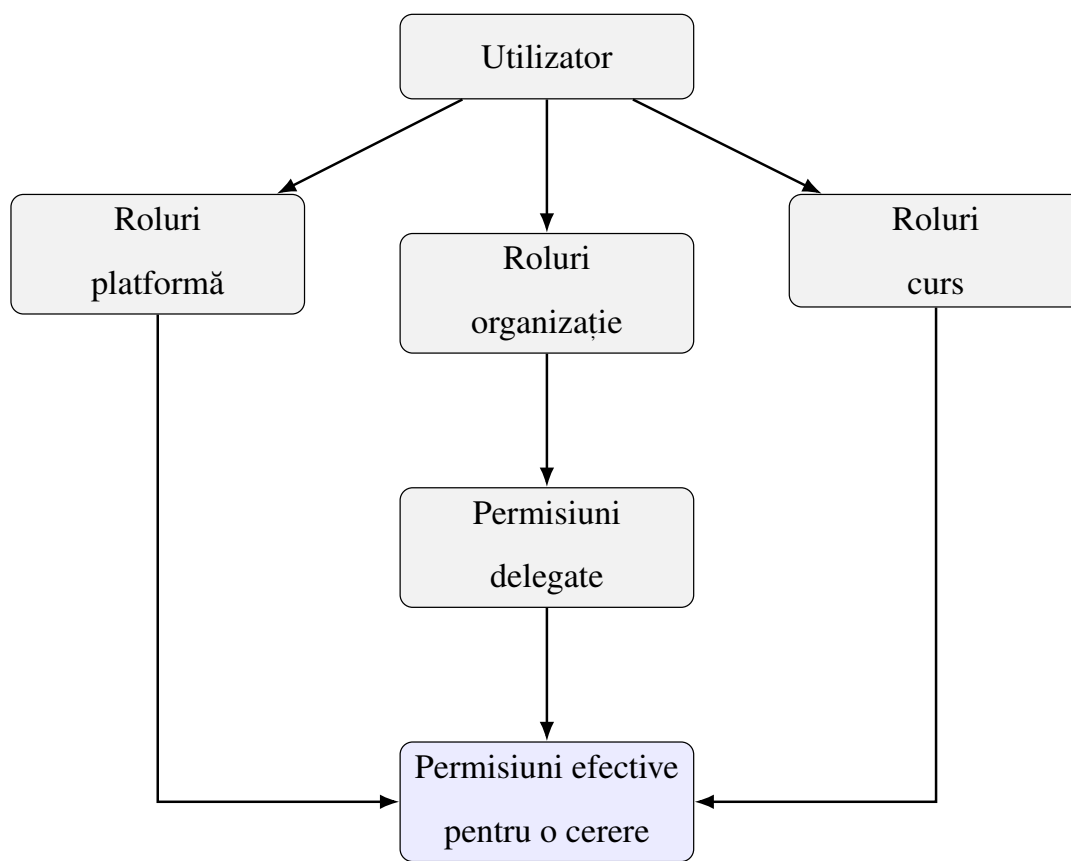


Figura 4.2. Model conceptual al permisiunilor pe scopuri (elaborare proprie).

4.8 Fluxul de recompensare

Fluxul de recompensare este proiectat ca un proces auditat. Un eveniment educațional creează un candidat la recompensă. Profesorul sau sistemul verifică evenimentul, platforma poate aproba suma, apoi se planifică plata, se înregistrează confirmarea tokenului, se creditează portofelul intern, se trimite notificarea și se rulează reconcilierea. Această succesiune previne dublarea recompenselor și permite intervenție umană atunci când apar erori sau suspiciuni.

Figura 4.3 prezintă stările principale ale fluxului. Unele etape pot fi automate, iar altele pot necesita aprobare. Valoarea arhitecturii constă în faptul că fiecare tranziție are un motiv și poate fi auditată. Dacă tranzacția blockchain este confirmată, dar creditarea portofelului intern eșuează, sistemul nu trimite încă o tranzacție. Reconcilierea pornește de la starea confirmată și completează etapa lipsă.

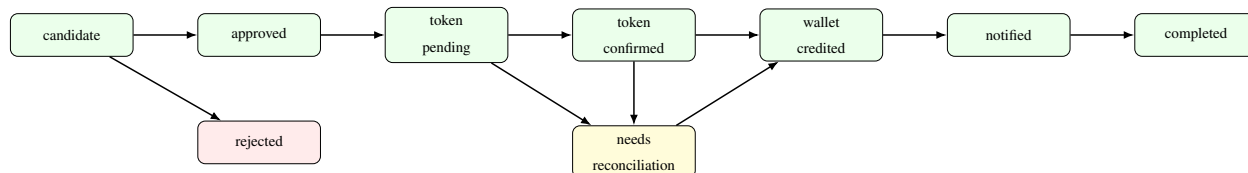


Figura 4.3. Fluxul de stări pentru recompensa LearnToken (elaborare proprie).

4.9 Persistența datelor

Schema PostgreSQL include tabele pentru utilizatori, autentificări, cursuri, capitole, conținut, progres, evaluări, organizații, roluri, permisiuni, notificări, portofele, tranzacții interne și externe, candidați la recompense, politici de recompensare, audit, fraud blocks, compensări și token burns. Această structură arată că platforma nu tratează recompensa ca o coloană simplă atașată cursului, ci ca un domeniu de sine stătător.

Persistența are două roluri. Primul este rolul operațional: aplicația trebuie să știe ce cursuri există, cine are acces, ce conținut este publicat și ce recompense sunt în așteptare. Al doilea este rolul de audit: platforma trebuie să poată explica de ce o recompensă a fost aprobată, cine a aprobat-o, ce politică era activă, ce tranzacție a fost creată și dacă portofelul a fost creditat. Pentru un sistem cu componente blockchain, baza de date nu concurează cu blockchain-ul, ci îl completează.

4.10 Valoarea arhitecturii

Arhitectura este valoroasă prin patru proprietăți. Prima este claritatea: fiecare strat are responsabilitate distinctă, iar fluxurile de business pot fi urmărite din HTTP către use case, domeniu și infrastructură. A doua este controlul: permisiunile pe scopuri și auditul permit administrare fină. A treia este reziliența: worker-ul, coada de joburi și reconcilierea reduc impactul eșecurilor parțiale. A patra este extensibilitatea: platforma poate adăuga noi tipuri de recompense, noi rute frontend, noi reguli de curs sau noi contracte fără a schimba fundamentul.

Valoarea practică apare mai ales în fluxurile în care educația și tokenizarea se întâlnesc. O finalizare de curs nu devine automat transfer on-chain; ea devine o decizie verificabilă. Un token nu este doar un număr într-o bază de date; el are o reprezentare prin contract. O eroare nu este doar un mesaj pierdut; ea produce o stare ce poate fi reconciliată. Această combinație face sistemul mai potrivit pentru o platformă educațională reală decât o arhitectură simplificată în care toate aceste operații sunt tratate informal.

4.11 Concluzie

RustLearn folosește o arhitectură hibridă și modulară pentru a susține cerințe educaționale și blockchain în același sistem. Backend-ul rămâne responsabil pentru reguli, permisiuni și fluxuri, baza de date păstrează starea și auditul, worker-ul procesează activități asincrone, iar Ethereum adaugă transparență pentru tokenuri. Această arhitectură oferă un echilibru între performanță, control și verificabilitate.

Capitolul 5

Implementarea platformei

5.1 Pornirea aplicației și compunerea stării

Implementarea backend-ului pornește de la un entrypoint subțire. Fișierul `src/main.rs` inițializează variabilele de mediu, logging-ul, starea aplicației și serverul Actix Web. Detaliile concrete de conectare la baza de date, S3, Ethereum și use case-uri sunt mutate în stratul `bootstrap`. Această decizie menține punctul de intrare ușor de citit și face ca inițializarea să poată fi testată sau refactorizată fără a modifica rutele HTTP.

Listing 5.1: Structura simplificată a pornirii serverului RustLearn.

```
let app_state = bootstrap::startup::initialize_app_state().await?;
HttpServer::new(move || {
    App::new()
        .configure(|cfg| bootstrap::app_data::configure_app_data(cfg, &
app_state))
        .configure(bootstrap::routes::configure_routes)
})
.bind("0.0.0.0:8080")?
.run()
.await
```

Starea aplicației include pool-ul PostgreSQL, starea S3, use case-urile pentru controlul accesului, conținut, identitate, KYC, învățare, notificări, organizații, raportare, recompense, aplicații de profesor și portofele. Această compunere este importantă pentru că API-ul nu creează dependențe ad-hoc în fiecare handler. În schimb, handler-ul primește dependența deja configurată, iar testarea poate înlocui porturile cu implementări controlate.

5.2 API-ul și middleware-ul

API-ul este expus sub `/api`. Scope-ul principal este protejat prin middleware JWT și middleware condițional. Rutele sunt configurate pe module: identitate, notificări, învățare, organizații, raportare, KYC, recompense, control acces, aplicații profesor și portofele. Această organizare reflectă domeniile sistemului și permite adăugarea de funcții noi fără a transforma fișierul principal de rute într-o listă greu de administrat.

Autentificarea cu JWT permite cererilor să transporte identitatea utilizatorului. Verificările de permisiuni sunt aplicate ulterior, în funcție de resursa accesată. De exemplu, o rută de recompense la nivel de curs trebuie să verifice permisiuni de curs, iar o rută de fraud block la nivel de platformă trebuie să verifice permisiuni de platformă. Această diferențiere este importantă deoarece același utilizator poate avea autoritate într-un context și niciun drept în altul.

5.3 Identitate, parole și verificare email

Fluxurile de identitate includ înregistrarea, autentificarea, politica de parolă, verificarea emailului, resetarea parolei și expunerea cheii publice prin JWKS. Politica de parolă solicită lungime minimă și diversitate de caractere, ceea ce reduce riscul unor parole triviale. Emailul de verificare este în prezent simulat local prin mesaje de dezvoltare, dar modelul aplicației separă deja generarea tokenului de transportul efectiv, permițând integrarea viitoare a unui furnizor real de email.

Expunerea JWKS este utilă pentru verificarea externă a tokenurilor. În loc ca toate componentele să cunoască cheia privată, aplicația poate publica cheia publică pentru validare. Cheile private rămân în variabile de mediu și nu sunt comise în repository. Această separare respectă principiul că secretele operaționale nu trebuie să fie parte din codul sursă.

5.4 Modelul cursurilor și al progresului

Domeniul de învățare include cursuri, capitole, conținut, progres și evaluări. Cursurile au stare de ciclu de viață, descriere, subiecte și prerechizite. Conținutul are stare de publicare, iar progresul este înregistrat pentru relația dintre utilizator, curs și material. Evaluările includ întrebări, punctaj, încercări și prag de promovare. Această structură permite platformei să distingă între simpla vizualizare a unui material și promovarea unei evaluări.

Pentru recompense, evaluarea produce un handoff către domeniul rewards numai dacă re-

zultatul este relevant. Dacă o încercare nu este promovată, nu se creează candidat la recompensă. Dacă este promovată, sistemul poate genera un eveniment de tip `assessment_completion`. Această abordare păstrează clară relația dintre performanța educațională și recompensa potențială.

5.5 Organizații și aplicații de profesor

Organizațiile permit gestionarea unor programe educaționale cu membri, cursuri și rapoarte proprii. Un utilizator poate fi invitat, poate primi roluri, poate accesa cursuri sponsorizate și poate vedea rapoarte în funcție de permisiuni. Aplicațiile de profesor sunt tratate ca un flux separat: utilizatorul depune cerere, organizația poate susține cererea, iar platforma poate aproba, respinge sau cere modificări. Această etapizare previne acordarea automată a drepturilor de creare și recompensare a cursurilor.

Implementarea include audit pentru deciziile privind aplicațiile de profesor. Fiecare schimbare de status poate fi urmărită, iar interfața frontend poate afișa istoricul. Această trasabilitate este importantă deoarece profesorii pot influența direct conținutul și pot aproba candidați la recompense.

5.6 Recompense și politici

Subdomeniul rewards este împărțit în use case-uri. Există module pentru trimiterea unui candidat, decizia profesorului, aprobarea sumei, planificarea plății, confirmarea tokenului, creditarea portofelului, notificarea utilizatorului, reconciliere, compensări, politici și blocaje antifraudă. Această granularitate face fluxul mai lung, dar mai sigur. Într-un sistem cu valoare economică, o implementare scurtă și implicită ar fi mai riscantă decât un proces auditat.

Politicile de recompensare definesc evenimente eligibile, sume și stări de activare. Ele permit schimbarea regulilor fără a modifica istoricul deciziilor vechi. Dacă o politică este modificată, auditul poate indica versiunea folosită la momentul aprobării. În plus, blocajele antifraudă pot opri recompense pentru profesori, organizații, cursuri sau politici suspecte. Această funcție este necesară pentru a evita ca tokenizarea să creeze un stimulent pentru abuz.

5.7 Portofele și tranzacții

Portofelele interne permit urmărirea valorii asociate unui utilizator sau unei organizații. Sistemul diferențiază între tranzacții interne, tranzacții externe și evenimente blockchain. Tranzacția internă este utilă pentru ledger-ul aplicației, iar tranzacția externă păstrează informații precum chain id, adresă contract, hash de tranzacție, adresă sursă și destinație. Prin această separare, platforma poate explica legătura dintre starea internă și evenimentele on-chain.

Fluxurile de depunere și retragere pot avea costuri de gas plătite de utilizator sau de platformă. Pentru scenariile în care platforma plătește gas, sistemul poate aplica o taxă configurabilă. Acest detaliu arată că integrarea blockchain nu este doar tehnică, ci și economică. Platforma trebuie să știe cine suportă costul operației și cum este înregistrată această decizie.

5.8 Contractele inteligente

Contractul `LearnToken` extinde ERC-20, suportă burn și permit și permite mint doar de către owner. Această regulă reduce riscul emiterii necontrolate. `LearnTokenPresigner` permite depunerea de tokenuri și execuția unor transferuri semnate, cu nonce și deadline pentru protecție împotriva replay. `PlatformImporter` permite importul tokenurilor prin permit EIP-2612, transferând fonduri către treasury sau către un destinatar specific.

Listing 5.2: Fragment conceptual al transferului semnat.

```
require(nonces[signer] == nonce, "invalid nonce");
bytes32 digest = _hashTypedDataV4(structHash);
address recovered = ECDSA.recover(digest, signature);
require(recovered == signer, "invalid signature");
nonces[signer] = nonce + 1;
```

Această logică este importantă pentru securitate. Nonce-ul împiedică reutilizarea aceleiași semnături, deadline-ul limitează durata de valabilitate, iar semnătura EIP-712 face cererea mai explicită pentru utilizator. Contractele nu rezolvă singure toate problemele de business, dar oferă o bază verificabilă pentru operațiile cu tokenuri.

5.9 Worker, procesare video și indexare

Worker-ul rulează separat de API. El preia joburi, procesează încărcări video, execută retry-uri și scrie heartbeat pentru health check. Pentru video, fluxul descarcă fișierul din S3, rulează `ffmpeg`, generează un MP4 procesat și un fișier audio, apoi încarcă rezultatele înapoi în bucket. Utilizatorul primește notificare de succes sau eșec. Această abordare respectă principiul că operațiile lungi trebuie mutate din calea critică a request-ului.

Worker-ul include și indexer pentru depuneri de tokenuri. El citește blocuri Ethereum, filtrează evenimentele relevante, evită dublarea evenimentelor importate și actualizează starea persistentă cu următorul bloc de scanat. Astfel, sistemul poate sincroniza portofelul intern cu transferuri on-chain observate, fără a cere utilizatorului să repete acțiuni manuale.

5.10 Frontend-ul produsului

Frontend-ul Next.js este organizat în suprafețe de produs. `/learn` este dashboard-ul cursantului, `/courses` afișează catalogul, `/rewards` prezintă istoricul recompenselor, iar `/wallet` gestionează portofelul. Suprafețele `/teach` permit profesorilor să gestioneze cursuri, conținut, înscrieri, studenți și recompense. Rutele `/organizations` oferă administrare pentru organizații, iar `/admin` acoperă operațiile de platformă.

Interfața derivă acțiunile disponibile din permisiuni, nu doar din numele rolului. Acest lucru este important pentru că rolurile pot fi directe sau delegate, iar un utilizator poate avea acces limitat într-un singur context. Un buton administrativ nu trebuie afișat doar pentru că utilizatorul are un rol textual, ci pentru că sesiunea curentă conține permisiunea necesară. Această disciplină reduce confuzia și previne apelarea unor operații care ar fi oricum respinse de backend.

5.11 Notificări și feedback operațional

Notificările oferă utilizatorului feedback pentru evenimente importante: procesare video, creditare wallet, recompense sau operații administrative. Într-o platformă educațională, notificările nu sunt doar elemente de confort. Ele reduc incertitudinea. Dacă o recompensă este întârziată, utilizatorul trebuie să vadă starea. Dacă un video nu poate fi procesat, profesorul trebuie să primească un mesaj clar. Dacă o aplicație de profesor este respinsă, auditul trebuie să păstreze motivul.

5.12 Concluzie

Implementarea RustLearn urmărește separarea clară a fluxurilor. Backend-ul expune API-uri protejate, use case-urile gestionează regulile, domeniul păstrează concepte stabile, infrastructura conectează PostgreSQL, S3 și Ethereum, iar frontend-ul oferă interfețe adaptate rolurilor. Contractele inteligente adaugă un strat verificabil pentru LearnToken, iar worker-ul asigură operații asincrone. Această implementare susține obiectivul lucrării: un sistem de eLearning în care recompensele blockchain sunt integrate controlat și auditat.

Capitolul 6

Validare, operare și limitări

6.1 Strategia de validare

Validarea platformei urmărește două direcții: validarea tehnică și validarea fluxurilor de produs. Validarea tehnică verifică dacă aplicația compilează, dacă migrațiile sunt coerente, dacă testele unitare și de integrare rulează, dacă frontend-ul trece testele și dacă serviciile locale pornesc în Docker Compose. Validarea fluxurilor de produs verifică scenarii precum autentificarea, accesul la cursuri, aprobarea profesorilor, administrarea recompenselor, creditarea portofelelor, raportarea și operarea worker-ului.

Repository-ul include comenzi Make pentru format, teste, preflight, testare în Compose, migrații și pornirea mediului de dezvoltare. Această alegere este importantă deoarece reduce dependența de pași manuali. În loc ca fiecare dezvoltator să ghicească ce comandă trebuie rulată, Makefile-ul devine interfața operațională a proiectului. Pentru o lucrare aplicativă, această disciplină arată că sistemul nu este doar cod demonstrativ, ci un produs care poate fi rulat și verificat.

6.2 Teste backend

Backend-ul are teste pentru domenii critice, inclusiv recompense, permisiuni, tokenuri, integrarea blockchain și adaptoare. Testele unitare verifică reguli izolate, cum ar fi parsarea tipurilor de evenimente token sau validarea unor stări de recompensare. Testele de integrare verifică fluxuri mai largi, unde baza de date, API-ul sau Ethereum local pot interacționa. Pentru contracte, testele Foundry verifică funcțiile LearnToken, LearnTokenPresigner și PlatformImporter.

Tabelul 6.1 sintetizează principalele niveluri de validare.

Tabela 6.1. Niveluri de validare pentru RustLearn (elaborare proprie).

Nivel	Ce verifică	Exemple de risc redus
Unit	Reguli de domeniu, validări, conversii de stare	Stări imposibile, parse greșit, calcule incorecte
Integrare API	Rute, middleware, permisiuni, persistență	Acces neautorizat, erori de contract API, date lipsă
Blockchain	Contracte, evenimente, semnături, deploy local	Replay, emitere necontrolată, ABI incompatibil
Frontend	Randare rute, stări goale, controale gated	Interfețe care afișează acțiuni fără permisiuni
Operațional	Health, readiness, Docker Compose, worker	Servicii pornite fără dependențe reale sau blocaje ascunse

6.3 Migrații și schema bazei de date

Migrațiile Diesel definesc evoluția bazei de date. Fiecare schimbare de schemă trebuie să aibă `up.sql` și `down.sql`, astfel încât evoluția să fie reversibilă când este posibil. Pentru o platformă cu recompense și portofele, migrațiile nu sunt doar detalii tehnice. Ele definesc structura de audit, tranzacții, stări și constrângeri de unicitate. De exemplu, cheile de idempotentă și indexurile unice reduc riscul dublării unor operații.

Schema curentă include tabele pentru candidați la recompense, politici, joburi de execuție, payout records, wallet credit records, compensation records, fraud blocks și audit events. Această separare permite interogări și raportări precise. Dacă toate aceste informații ar fi păstrate într-un singur câmp JSON, dezvoltarea inițială ar fi mai rapidă, dar verificarea și reconcilierea ar deveni mai dificile.

6.4 Health și readiness

RustLearn expune `/health` pentru liveness și `/ready` pentru dependențe. Diferența este importantă. Un proces poate fi viu, dar nepregătit să servească trafic dacă baza de date, S3 sau Ethereum nu sunt disponibile. Readiness-ul verifică dependențele înainte ca platforma să fie considerată

operațională. Frontend-ul poate afișa aceste stări în dashboard-ul de administrare, iar Kubernetes sau Docker pot folosi endpoint-urile pentru health checks.

Worker-ul are propriul mecanism de heartbeat. El scrie periodic un fișier care poate fi verificat de health check. Dacă worker-ul se blochează, lipsa heartbeat-ului indică problema. Această verificare este utilă deoarece worker-ul nu expune neapărat un server HTTP, dar este critic pentru procesarea video și indexarea depunerilor.

6.5 Docker Compose și mediu local

Mediul local include serviciile `app`, `web`, `db`, `rustfs`, `anvil`, `worker` și `test-runner`. PostgreSQL rulează ca bază de date, RustFS oferă stocare compatibilă S3, Anvil simulează rețeaua Ethereum, iar worker-ul rulează sarcinile asincrone. Această compunere permite testarea locală a fluxurilor care, în producție, ar depinde de servicii externe.

Valoarea Docker Compose este reproducibilitatea. O platformă care integrează DB, S3, blockchain și worker poate deveni greu de pornit manual. Compose definește dependențele, variabilele și porturile într-un singur loc. În plus, același model conceptual poate fi transpus în Kubernetes, unde readiness-ul, secretele, volumele și restart policy devin elemente de operare.

6.6 Securitate și control al riscurilor

Riscurile principale sunt accesul neautorizat, dublarea recompenselor, manipularea portofelelor, expunerea secretelor și inconsistența între baza de date și blockchain. Sistemul reduce aceste riscuri prin JWT, permisiuni pe scopuri, audit, stări de recompensare, politici versionate, nonce-uri în contracte, deadline-uri pentru semnături, chei de idempotentă și reconciliere.

Un risc specific aplicațiilor blockchain este tentația de a considera blockchain-ul suficient pentru audit. În realitate, blockchain-ul poate confirma o tranzacție, dar nu explică toate deciziile educaționale care au dus la ea. De aceea, RustLearn păstrează auditul deciziilor în PostgreSQL și leagă tranzacțiile externe de candidați, portofele și politici. Această legătură este necesară pentru investigații și rapoarte.

6.7 Limitări ale implementării

Implementarea curentă are limitări. În primul rând, unele fluxuri frontend marchează explicit funcții viitoare, cum ar fi progresul persistent complet, încărcarea media avansată sau anumite detalii de buget. În al doilea rând, emailul este încă simulat local, nu integrat cu un furnizor real. În al treilea rând, contractele nu sunt upgradeable proxies, ceea ce înseamnă că schimbările majore de ABI necesită redeploy și plan de reconciliere. În al patrulea rând, utilizarea tokenurilor în producție ar necesita audit de securitate dedicat și politici juridice clare.

O altă limitare este lipsa unei evaluări cu utilizatori reali. Lucrarea validează arhitectura și implementarea tehnică, dar nu măsoară încă impactul recompenselor asupra motivației sau rezultatelor educaționale. Pentru o cercetare completă de produs, ar fi necesar un studiu cu grupuri de cursanți, comparație între cursuri cu și fără recompense și măsurarea retenției, promovării și satisfacției.

6.8 Scenarii de testare recomandate

Pentru verificare completă, sunt recomandate următoarele scenarii:

- înregistrarea unui cursant, verificarea emailului, autentificarea și accesarea dashboard-ului `/learn`;
- aplicarea pentru rol de profesor, aprobarea de către platformă și accesarea rutelor `/teach`;
- crearea unui curs, adăugarea de capitole și conținut, publicarea și solicitarea de înscriere de către cursant;
- completarea unei evaluări, generarea candidatului la recompensă și aprobarea de către profesor;
- aprobarea sumei la nivel de platformă, planificarea plății și creditarea portofelului;
- încărcarea unui video și verificarea procesării prin worker;
- simularea unei stări inconsistente și rularea reconcilierii;
- verificarea interfețelor fără permisiuni, pentru a confirma că acțiunile sensibile nu sunt afișate sau executate.

6.9 Concluzie

Validarea RustLearn trebuie privită ca un proces continuu. Sistemul are mecanisme tehnice solide: teste, migrații, health checks, readiness checks, worker heartbeat și mediu local compus. Totuși, integrarea blockchain introduce riscuri suplimentare, iar trecerea în producție ar necesita audit de securitate, monitorizare, backup-uri și politici clare de operare. Arhitectura existentă creează baza pentru aceste etape, deoarece stările, auditul și separarea responsabilităților sunt deja prezente.

Capitolul 7

Concluzii și direcții viitoare

7.1 Sinteza lucrării

Lucrarea a avut ca obiectiv proiectarea și implementarea unui sistem de eLearning bazat pe blockchain, în limba română, conform cerințelor academice pentru o disertație. Tema a fost abordată prin analiza domeniului eLearning, a gamificării, a modelului learn-to-earn și a aplicațiilor hibride Web2/Web3, apoi prin proiectarea și documentarea platformei RustLearn. Rezultatul este un sistem care combină un backend Rust/Actix Web, o bază PostgreSQL, stocare S3, worker asincron, contracte Solidity și frontend Next.js.

Concluzia principală este că blockchain-ul aduce valoare într-o platformă educațională atunci când este folosit selectiv. Nu toate datele educaționale trebuie puse on-chain. Lecțiile, progresul, permisiunile, evaluările, rapoartele și auditul operațional sunt mai potrivite într-o infrastructură controlată, rapidă și relațională. În schimb, tokenurile, transferurile și evenimentele financiare verificabile pot beneficia de transparența și interoperabilitatea Ethereum. Această separare este esențială pentru caracterul practic al soluției.

7.2 Contribuții realizate

Prima contribuție este definirea unei arhitecturi hibride pentru eLearning recompensat. Arhitectura arată cum pot coexista un sistem centralizat de cursuri și un strat blockchain de recompense fără ca unul să îl compromită pe celălalt. Backend-ul rămâne sursa de adevăr pentru reguli, iar blockchain-ul devine strat de verificare și transfer pentru tokenuri.

A doua contribuție este modelarea recompensei ca proces auditat. În loc să se acorde automat tokenuri la primul eveniment, RustLearn folosește candidați, politici, aprobări, planificare

de payout, confirmare token, creditare wallet, notificare și reconciliere. Acest model este mai potrivit pentru o platformă reală deoarece acceptă erori, întârzieri și decizii administrative fără a pierde trasabilitatea.

A treia contribuție este separarea permisiunilor pe scopuri: platformă, organizație și curs. Această separare reflectă mai bine realitatea educațională decât un model global simplu. Un utilizator poate avea rol de profesor într-un curs, rol de membru într-o organizație și rol de cursant în alt context. Arhitectura permite exprimarea acestor situații fără privilegii excesive.

A patra contribuție este integrarea operațională a componentelor. Sistemul include migrații, health checks, readiness checks, Docker Compose, worker heartbeat, teste și documentație de operare. Aceste elemente fac diferența între un prototip izolat și o platformă care poate fi rulată, verificată și extinsă.

7.3 Răspuns la problema inițială

Problema inițială a fost construirea unui sistem de eLearning bazat pe blockchain care să fie util, sigur și justificat arhitectural. Soluția propusă răspunde acestei probleme printr-o arhitectură modulară monolitică și hibridă. Monolitul modular reduce complexitatea operațională, iar separarea internă păstrează domeniile clare. Integrarea blockchain este limitată la tokenuri și fluxuri de valoare, ceea ce evită costuri inutile și protejează datele educaționale.

Tehnologiile alese sunt coerente cu această problemă. Rust oferă siguranță și performanță, Actix Web oferă un API modular, PostgreSQL și Diesel oferă consistență relațională, S3 susține conținut multimedia, worker-ul separă operațiile costisitoare, Next.js oferă suprafețe de produs clare, iar Solidity/OpenZeppelin oferă contracte tokenizate mature. Fiecare tehnologie are un rol justificat în ansamblu.

7.4 Limitări

Lucrarea are limite care trebuie recunoscute. În primul rând, nu include un studiu cu utilizatori reali pentru a măsura impactul recompenselor asupra motivației. În al doilea rând, unele fluxuri sunt încă pregătite pentru dezvoltare viitoare, în special emailul real, media upload avansat și unele detalii de progres persistent. În al treilea rând, contractele inteligente ar necesita audit extern înainte de utilizare în producție cu valoare reală. În al patrulea rând, orice sistem learn-to-earn trebuie analizat și din perspectivă juridică, fiscală și etică.

Aceste limitări nu invalidează soluția, ci indică diferența dintre o platformă aplicativă de disertație și un produs complet lansat public. Arhitectura a fost concepută tocmai pentru a permite aceste extinderi fără schimbări fundamentale.

7.5 Direcții viitoare

O direcție viitoare este introducerea unui serviciu real de email, cu livrare, template-uri, retry și monitorizare. O altă direcție este extinderea progresului persistent, astfel încât fiecare lecție, activitate și evaluare să contribuie la un model mai bogat de învățare. De asemenea, platforma poate adăuga analytics educațional pentru profesori și organizații, cu indicatori privind retenția, dificultatea lecțiilor și rata de promovare.

Pe partea blockchain, direcțiile viitoare includ audit extern pentru contracte, suport pentru rețele de test publice, mecanisme de treasury mai avansate, politici de emisie mai stricte și interfețe mai clare pentru MetaMask sau alte portofele. Reconcilierea poate fi extinsă cu joburi programate, alerte și dashboard-uri pentru operatori. În plus, token burns și taxele de depunere sau retragere pot fi integrate într-un model economic mai formal.

Pe partea de produs, este necesar un studiu cu utilizatori reali. Platforma poate fi testată pe un set de cursuri pilot, comparând grupe cu recompense tokenizate și grupe fără recompense. Indicatorii relevanți ar putea include rata de finalizare, numărul de reveniri, rezultatele evaluărilor și percepția cursanților asupra utilității recompenselor. Acest studiu ar transforma lucrarea dintr-o contribuție tehnică într-o cercetare aplicată asupra impactului motivațional.

7.6 Concluzie finală

Sistemul de eLearning bazat pe blockchain prezentat în lucrare demonstrează că tokenizarea poate fi integrată într-o platformă educațională fără a sacrifica cerințele clasice de securitate, performanță și control. Valoarea soluției nu stă în folosirea blockchain-ului ca scop în sine, ci în folosirea lui acolo unde aduce transparență și verificabilitate. Prin arhitectura hibridă, RustLearn oferă o bază solidă pentru cursuri, evaluări, organizații, portofele și recompense LearnToken, păstrând posibilitatea de extindere către scenarii reale de învățare recompensată.

Bibliografie

- [1] V. Bell, C. Silva, J. A. Guina și T. Fernandes, „Community Health of Children and Adolescents in Sub-Saharan Africa”, în *European Journal of Medical and Health Sciences* 5 (Iun. 2023), pp. 22–31, DOI: 10.24018/ejmed.2023.5.3.1702.
- [2] G. Coulouris, J. Dollimore și T. Kindberg, *Distributed Systems: Concepts and Design*, 3rd, Pearson, Addison Wesley, 2001.
- [3] V. P. Dennen, B. M. Hall și A. Hedquist, „Instructional Strategies for Engaging Online Learners: Do Learner-centeredness and Modality Matter?”, în *Online Learning Journal* 27.1 (2022), pp. 158–186, DOI: 10.24059/olj.v27i1.3430.
- [4] S. Deterding, M. Sicart, L. Nacke, K. O’Hara și D. Dixon, „Gamification: Using Game Design Elements in Non-Gaming Contexts”, în *Proceedings of the 2011 Annual Conference on Human Factors in Computing Systems*, 2011, pp. 2425–2428, DOI: 10.1145/1979742.1979575, URL: <https://dl.acm.org/doi/10.1145/1979742.1979575>.
- [5] L. Freina și M. Ott, „A literature review on immersive virtual reality in education: State of the art and perspectives”, în *The International Scientific Conference eLearning and Software for Education*, vol. 1, ”Carol I” National Defence University, 2015, pp. 133–141.
- [6] S. Hrastinski, „A theory of online learning as online participation”, în *Computers & education* 52.1 (2009), pp. 78–82.
- [7] S. Klabnik și C. Nichols, *The Rust Programming Language*, No Starch Press, 2019, URL: <https://doc.rust-lang.org/book/>.
- [8] M. Liu, Y. Li și S. Li, „Web3: Exploring Decentralized Technologies and Applications for the Future of Empowerment and Ownership”, în *Blockchains* 1.2 (2023), pp. 111–131, DOI: 10.3390/blockchains1020008.

- [9] F. Martin și D. U. Bolliger, „Engagement matters: Student perceptions on the importance of engagement strategies in the online learning environment”, în *Online Learning Journal* 22.1 (2018), pp. 205–222, DOI: 10.24059/olj.v22i1.1092, URL: <https://doi.org/10.24059/olj.v22i1.1092>.
- [10] C. Paans, I. Molenaar, E. Segers și L. Verhoeven, „Temporal variation in children’s self-regulated hypermedia learning”, în *Computers in Human Behavior* 84 (2018), pp. 307–319, DOI: 10.1016/j.chb.2018.04.002, URL: <https://doi.org/10.1016/j.chb.2018.04.002>.
- [11] J. Radianti, T. A. Majchrzak, J. Fromm și I. Wohlgenannt, „A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned, and research agenda”, în *Computers & Education* 147 (2020), p. 103778.
- [12] K. Squire, *Video games and learning: Teaching and participatory culture in the digital age*, Teachers College Press, 2011.

Webografie

- [13] Amazon Web Services, *Amazon Simple Storage Service Documentation*, 2026, URL: <https://docs.aws.amazon.com/s3/> (accesat în 17.6.2026).
- [14] BitDegree, *BitDegree: Online Learning Platform*, 2024, URL: <https://www.bitdegree.org> (accesat în 17.6.2026).
- [15] Brave, *Introducing the Brave Wallet, a Browser-Native Crypto Wallet With No Extension Required*, 2021, URL: <https://brave.com/blog/brave-wallet-launch/> (accesat în 17.6.2026).
- [16] V. Buterin, *Ethereum Whitepaper*, 2014, URL: <https://ethereum.org/whitepaper/> (accesat în 17.6.2026).
- [17] CoinCodex, *Brave vs Chrome: Should You Switch to Crypto-Friendly Browser?*, 2024, URL: <https://coincodex.com/article/36577/brave-vs-chrome/> (accesat în 17.6.2026).
- [18] Diesel Project, *Diesel: Getting Started*, 2026, URL: <https://diesel.rs/guides/getting-started/> (accesat în 17.6.2026).

- [19] N. Johnson, F. Araoz, M. Evans, M. Lundkvist și A. Tomaino, *EIP-712: Typed Structured Data Hashing and Signing*, 2017, URL: <https://eips.ethereum.org/EIPS/eip-712> (accesat în 17.6.2026).
- [20] S. Klabnik, C. Nichols, C. Kryocho și T. R. Community, *The Rust Programming Language*, Rust Project, 2026, URL: <https://doc.rust-lang.org/book/> (accesat în 17.6.2026).
- [21] Moralis, *Decentralized Applications Explained: What Are dApps?*, 2024, URL: <https://moralis.io/decentralized-applications-explained-what-are-dapps/> (accesat în 17.6.2026).
- [22] D. Novac, *The Basics of UI Design: Creating Intuitive User Interfaces*, UX Media, 2023, URL: <https://uxmedia.io/blog/what-is-user-interface-ui-design/> (accesat în 17.6.2026).
- [23] OpenZeppelin, *ERC-20 Documentation*, 2026, URL: <https://docs.openzeppelin.com/contracts/5.x/erc20> (accesat în 17.6.2026).
- [24] P. Pandya, *The Fundamentals of UI and UX Design: A Comprehensive Guide*, The Cuneiform, 2024, URL: <https://www.thecuneiform.com/insights/the-fundamentals-of-ui-and-ux-design-a-comprehensive-guide/> (accesat în 17.6.2026).
- [25] Phemex, *Learn Crypto: Educational Resources and Rewards*, 2024, URL: <https://phemex.com/learn-crypto> (accesat în 17.6.2026).
- [26] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2026, URL: <https://www.postgresql.org/docs/> (accesat în 17.6.2026).
- [27] Solidity Project, *Solidity Documentation*, 2026, URL: <https://docs.soliditylang.org/> (accesat în 17.6.2026).
- [28] The Actix Team, *Actix Web Documentation*, 2026, URL: <https://actix.rs/docs/> (accesat în 17.6.2026).
- [29] Uniswap și V. Buterin, *ERC-2612: Permit Extension for EIP-20 Signed Approvals*, 2020, URL: <https://eips.ethereum.org/EIPS/eip-2612> (accesat în 17.6.2026).
- [30] Vercel, *Next.js Docs: App Router*, 2026, URL: <https://nextjs.org/docs/app> (accesat în 17.6.2026).